



NAVAL
POSTGRADUATE
SCHOOL

AE2440: Assignment 1 and Loops / Vectors Intro



Plan of the Day

- Review Assignment 1
- Bike Share Discussion
 - Underlying model type
 - Introduction to loops and vectors
- Iteration Intro



Overview of Assignment

- `penny.m`: First script to use MATLAB for simple calculations
- `pennywithair.m`: First live script to combine simple calculations with rich text (and an equation)
- `bike_share.m`: Variable assignment. Memory and storage: using temporary variable(s). Introduces a series and convergence.
- `aquarium.m`: More MATLAB-as-a-calculator. Creating a habit of documenting units to avoid semantic errors.



General Observations

- Simple output is sufficient for now.
 - MATLAB's unique behavior of displaying output in Command Window
 - We'll learn more sophisticated formatting as we go.
- Naming of files is important:
 - `bike_update.m` $\neq$ `bikeupdate.m`
 - `Penny.m` $\neq$ `penny.m`
- See ``comments.txt`` in Sakai



Pennywithair.m

- Live Script editor
 - Adding equation objects
 - Adding images

Now that we have our variables initialized, the first step will be to calculate how long it will take the penny to reach its terminal velocity. We solve this by taking the terminal velocity of the penny, divided by the acceleration from gravity.

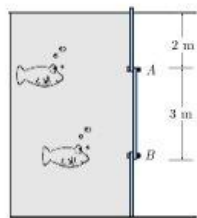
$$t_1 = \frac{(v_t - v_0)}{a}$$

$$t_2 = \sqrt{\frac{2 * (h_{\text{terminal}})}{a}}$$

Aquarium

This script solves the following statics problem:

An aquarium tank has a window AB for viewing the inhabitants. The width of the window is fixed at $w = 1.5$ m. The top of the window is shown as 2 m below the surface, but should be parameterized as the variable a . The height of the window is shown as 3 m, but should be considered variable b . The tank contains water with a density of 1000 kg/m^3 .



Pressure at Top of Window

$$P_A = p g(2m) = 19620 \frac{N}{m^2}$$

Pressure at Bottom of Window

$$P_B = p g(5m) = 49050 \frac{N}{m^2}$$

$$P_{\text{avg}} = \frac{(P_A + P_B)}{2}$$



bike_update.m

- As you read, this is a lead-in to discussing iteration.
- What is the underlying model?

```
pg = pg + round(m * 0.05) - round(pg * 0.03)
m = m + round(pg * 0.03) - round(m * 0.05)
```

```
m = (m - (m * 0.05)) + (pg * 0.03);
m = round(m)
pg = (pg - (pg * 0.03)) + (m * 0.05);
pg = round(pg)
```

```
9
10
11
12
```

```
m = m - round(m * 0.05) + round(p * 0.03)
```

```
p = p - round(p * 0.03) + round(m * 0.05)
```

```
1 %% PPM Exercise 2.2
2
3 % Need to define the following the variables in the command window
4 % pg = 100; % initializing starting number of bikes in PG
5 % m = 100; % initializing starting number of bikes in Monterey
6 % day = 1; % initializing day counter
7
8 mmoved = round(m * (0.05)) % bikes moved out of Monterey
9 pgmoved = round(pg * (0.03)) % bikes moved out of Pacific Grove
10
11 m = m - mmoved + pgmoved; % new number of bikes in Monterey
12 pg = pg - pgmoved + mmoved; % new number of bikes in Pacific Grove
13
14 day = day + 1 % update day counter
15
16 % the bikes stabilize at 83 bikes in Monterey and 117 bikes in Pacific
17 % Grove after 12 days
```



Iteration via `for` loops

For Loop

Iteratively execute child components

R2024b

Description

This component functions like the MATLAB® `for` loop, except that instead of executing a statement, it executes its child components. It must have at least one child component to execute.

Loop Type

The loop type can have incremented indices or a vector of indices. For more information on `for` loops and indices, see [for](#).

- **Incremented indices:** Executes a `for` loop of the form:

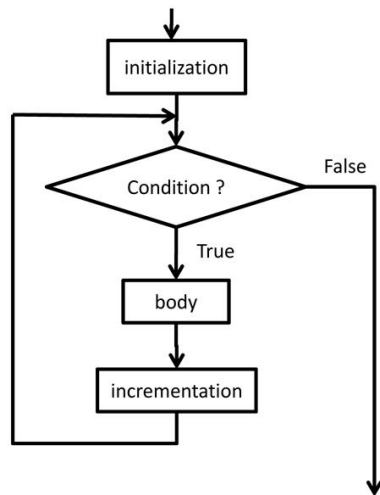
```
for varname=x:y:z
```

- **Start:** Corresponds to `x` in the previous expression.
- **Increment:** Corresponds to `y` in the previous expression.
- **End:** Corresponds to `z` in the previous expression.

- **Vector of Indices:** Executes a `for` loop of the form:

```
for varname=[a b c ...]
```

Specify appropriate values in the **Vector** field in the form `a b c ...`





Iteration via `for` loops - Examples

For Loop

```
for jj = 1:10  
    disp(jj)  
end
```

or loop, except that it is a component to execute.

```
for nn = 10:3:15  
    disp(nn)  
end
```

R2024b

Loop Type

The loop type can have incremented indices or a vector of indices. For more information on for loops and indices, see [for](#).

- **Incremented indices:** Executes a for loop of the form:

```
for varname=x:y:z
```

- **Start:** Corresponds to x in the previous expression.
- **Increment:** Corresponds to y in the previous expression.
- **End:** Corresponds to z in the previous expression.

- **Vector of Indices:** Executes a for loop of the form:

```
for varname=[a b c ...]
```

Specify appropriate values in the **Vector** field in the form a b c ...

```
sum = 0;  
for ii = 1:5  
    sum = sum + ii;  
end  
disp(sum)
```

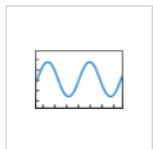
```
graph TD; A[for ii = 1:5] --> B[sum = sum + ii;]; B --> C[end]; C --> D[disp(sum)];
```



Using plot

plot

2-D line plot



Syntax

```
plot(X,Y)
plot(X,Y,LineStyle)
```

Description

Vector and Matrix Data

`plot(X,Y)` creates a 2-D line plot of the data in Y versus the corresponding values in X.

- To plot a set of coordinates connected by line segments, specify X and Y as vectors of the same length.
- To plot multiple sets of coordinates on the same set of axes, specify at least one of X or Y as a matrix.

`plot(X,Y,LineStyle)` creates the plot using the specified line style, marker, and color.

▼ LineSpec — Line style, marker, and color

string scalar | character vector

Line style, marker, and color, specified as a string scalar or character vector containing symbols. The symbols can appear in any order. You do not need to specify all three characteristics (line style, marker, and color). For example, if you omit the line style and specify the marker, then the plot shows only the marker and no line.

Example: "--or" is a red dashed line with circle markers.

Line Style	Description	Resulting Line
"_"	Solid line	————
"--"	Dashed line	- - - - -
"."	Dotted line	
"-."	Dash-dotted line	- . - . - .

Marker	Description	Resulting Marker
"o"	Circle	○
"+"	Plus sign	+
"*"	Asterisk	*
"."	Point	•
"x"	Cross	×



Exercise 3.1. Create a live script named *bike_loop.mlx* that does the following:

1. Assign the initial values (precondition) of `m = 100` and `pg = 100`.
2. Run the `for` loop from 1 to 30.
 - (a) Inside the loop, call the program *bike_update.m* to update the values of `m` and `pg`.
The program should echo the values of `m` and `pg` after each update.

If everything goes smoothly, your script will display a long stream of numbers on the screen. It's probably too long to fit, and even if it did fit, it would be hard to interpret. A graph (coming soon) would be much better!



Exercise 3.2. Using *bike_loop.mlx* as a starting point, develop a live script —
- *bike_loop_plot.mlx* so that it clears the figure before running the loop. Then, each time
through the loop, it should plot the value of *m* versus the value of *i* with a red circle.

Once you get that working, modify it so it plots the values of *pg* with blue diamonds.



Next Steps

- Tuesday: Read PMM Ch4: Vectors
 - Vectors and Loops
- Wednesday: Take a look at Assignment 2 prior to class
 - Bring computers